Contents

The Cozy Manual **1**
Contents . . . 1
1. Getting started . . . 2
2. The REPL . . . 2
3. Values and types . . . 4
4. Operators and expressions . . . 5
5. Variables and scope . . . 6
6. Control flow . . . 6
7. Functions . . . 7
8. Arrays and matrices . . . 9
9. Linear algebra . . . 10
10. Complex numbers . . . 11
10b. Dual numbers and exact derivatives . . . 11
11. Special functions and statistics . . . 11
12. Random numbers . . . 14
13. Plotting . . . 14
14. Data files . . . 15
15. Output and formatting . . . 15
15b. Strings as code, and the keyboard (since 2.19) . . . 16
16. Scripts and tools . . . 18
Editors . . . 18
17. Builtin reference . . . 19
18. Grammar summary . . . 26

The Cozy Manual

A small functional array language — user manual for the language, the REPL, and the tools.

This manual covers Cozy as implemented: every example below was executed against the interpreter and shows its actual output. For a quick pitch and build instructions see the README; for the honest list of sharp edges see KNOWN_LIMITATIONS.

Contents

1. Getting started
2. The REPL
3. Values and types
4. Operators and expressions
5. Variables and scope
6. Control flow
7. Functions
8. Arrays and matrices
9. Linear algebra
10. Complex numbers — Dual numbers and exact derivatives
11. Special functions and statistics
12. Random numbers
13. Plotting
14. Data files
15. Output and formatting
16. Scripts and tools — Editors
17. Builtin reference
18. Grammar summary

1. Getting started

Build (a C23 compiler and `libm`; readline optional — see the README for macOS notes):

```
make -j$(nproc)   # ./cozy, the REPL (parallel; plain 'make' works too)
make test         # golden suite + codegen goldens
```

The banner shows the version, when the binary was built, and when the session started. `cozy --version` prints the same from the command line. Start the REPL and type an expression; its value echoes back:

```
cozy> 2 + 3 * 4
14
cozy> [1, 2; 3, 4] * [5; 6]
[ 17
   39 ]
```

A trailing `;` evaluates a statement but suppresses the echo. `#` and `%` both start a comment that runs to end of line. Ctrl-D exits; Ctrl-C cancels the current input.

2. The REPL

The interactive shell adds conveniences on top of the language. None of them apply to scripts — they are REPL features.

Commands (typed bare, not as function calls):

Command	Effect
`help` / `help(f)`	catalogue of all builtins / details plus usage examples for one
`who`	your variables, with type and shape; `who("records")`, `who("functions")`, `who("vars")`, add `"sorted"`
`whov` / `whof` / `whor` / `whos`	shorthands: vars, functions, records, everything-sorted
`version`	the interpreter version, as a string
`clear()` / `clear("a", ...)`	remove all user variables / the named ones (builtins are safe)
`keep("a", ...)`	remove all user variables *except* the named ones (the complement of `clear`)
`mem`	workspace size and peak process memory
`format …`	number display: `format("fixed", 2)`, `format(8)` (significant), `format long`, `format` to show
`pretty on|off`	aligned multi-line matrix display (default on in the REPL)
`manual [doc]`	page a rendered document: `manual`, `manual packages`, `manual changelog`, `manual lessons`, `manual design`
`TAB`	complete builtins, your names, and keywords; inside a `"quoted string"` it completes file paths
`more on|off`	page long output through `$PAGER` (default off)
`!cmd`	run a shell command (`!ls`, `!git status`)
`dis(f)`	disassemble a function's bytecode

`keep` names what survives; everything else goes, standard library untouched:

```
cozy> let rate = 0.0575; let n = 360; let scratch = 99; let tmp = [1, 2, 3];
cozy> keep("rate", "n"); who
  rate         float      = 0.0575
  n            int        = 360
```

Every builtin's help includes executable examples with their actual output — the examples are machine-verified against the interpreter, like this manual:

```
cozy> help(median)
  median(A) | median(A, dim)
      median of all elements, or along dim
      builtin, 1 to 2 arguments
      e.g.  median([1, 2, 3, 4])              % 2.5
```

Autocall. A bare name that is a zero-argument builtin or closure is called: `who`, `help`, `rand` work without parentheses, and so does your own `let f = fn -> 42; f`.

Line editing. With readline (or macOS libedit), you get history (persisted to `~/.cozy_history`), completion on builtin and variable names, and the usual Emacs keys. Multi-line constructs continue with a `...>` prompt until the `end` arrives.

Display defaults. The REPL prints matrices as aligned blocks (`pretty on`) and numbers in a terse 6-significant-digit format; both are configurable — see Output and formatting.

The working directory. `pwd`, `cd("dir")`, and `ls` are ordinary builtins, so they work bare at the prompt (shell muscle memory intact) *and* as values: `ls` returns a string array you can pipe, and `cd` actually changes the interpreter's directory — which `!cd` cannot (the shell escape runs in a child process). Globs work: `ls("*.cz")`. Flags belong to the shell escape: `!ls -la`.

```
cozy> ls("packages")
["astro.cz"; "autodiff.cz"; "demo.cz"; "dist.cz"; "finance.cz"; "optim.cz"; "phys.cz"; "poly.cz"; "rmt.c
cozy> cd("packages");
cozy> load("dist.cz"); norm.cdf(0, 0, 1)
0.5
cozy> cd("..");
```

`ans` — the last value you saw and didn't name. Every echoed expression statement rebinds `ans`; `let` statements don't (their result has a name), semicolon-suppressed statements don't, and `load()`ed scripts don't (they don't echo). One rule: *if a value printed without a name, it's in `ans`* — the screen is the spec. `ans` is an ordinary global: it shows in `who`, `clear("ans")` removes it, and a `let ans = ...` of your own is legal and simply gets overwritten by your next anonymous result.

```
cozy> 3 + 4
7
cozy> ans * 2
14
cozy> sqrt(ans)
3.74166
cozy> let named = ans
3.74166
cozy> 9 * 9
81
cozy> let x = 1
1
cozy> ans
81
cozy> ans + 100;
cozy> ans
81
```

```
cozy> s * 2 where s = 50
100
cozy> ans
100
cozy> who
  ans           int         = 100
  named         float       = 3.74166
  x             int         = 1
```

Note the quiet guarantees in that transcript: `ans` survived both the `let` and the suppressed statement — it can never hold something you didn't see — and a `where`-qualified expression sets it like any other, because a where clause is an anonymous expression, not a named binding. This differs from Octave, where `3 + 4;` sets `ans` silently and scripts clobber it as a side effect.

3. Values and types

Identifiers are `[A-Za-z_]` plus any UTF-8 byte above ASCII, then digits too — so Greek reads naturally: `let  = 0.05`, `let  = [1; 2]`, `{  = 1,  = 2}.` , `minimize( , 0)`. Names compare by *bytes*: two visually identical characters in different Unicode normalizations are different names (type your one way). Keywords stay ASCII.

Cozy has thirteen value kinds. The scalar kinds:

Type	Literals	Notes
`Int`	`42`, `-7`	64-bit signed, exact; overflow PROMOTES to Float (3^84 is 1.19725e+40, not a wrapped integer)
`Float`	`3.14`, `1e-9`, `2.5e3`	IEEE double
`Bool`	`true`, `false`	distinct from numbers: `1 == true` is an error
`Complex`	`2i`, `1 + 3i`, `2.5i`	double re/im pair
`Dual`	`dual(2, 3)`, prints `2+3eps`	value + derivative pair, `eps^2 = 0`; see the dual numbers section
`HDual`	`hdual(3, 1, 1)`, prints `3+1eps1+1eps2+0eps12`	hyper-dual: two nilpotent directions whose product survives, so one pass carries an exact second derivative in the `eps12` slot; `hdual12` reads it — the engine under `hess` and `minimize_newton`
`Date`	`date("2024-03-15")`, prints `2024-03-15`	a calendar date: days since 1970-01-01 UTC in a double (fractional time-of-day); strict algebra — `Date + number` is a Date, `Date - Date` is days, anything else errors; `readtable` auto-types ISO columns
`String`	`"hello"`	byte strings: `+` concatenates, comparisons are lexicographic, `s[i]`/`s[a:b]` index bytes (see the Strings section)
`Null`	`null`	the "no value" value; a suppressed or valueless statement yields it

And the compound kinds: `Array` (the 2-D numeric matrix — every array is rows x cols; a scalar is *not* a 1x1

array), `Record` (`{x = 1, y = 2}`, fields via `.x`), `Function` (builtins and closures), and `Sparse` (CSR sparse matrices — see the sparse matrices section).

The type discipline is strict rather than coercive: `Bool` does not silently become a number in arithmetic or comparison (`1 == true` errors), and mixing `Bool` with numerics in an array literal is an error. The one deliberate blur: `Int` and `Float` compare and combine freely (`5 == 5.0` is `true`), and `/` always produces a `Float` (`4 / 2` is `2.0`).

Floating point behaves like floating point:

```
cozy> 0.1 + 0.2 == 0.3
false
cozy> 5 / 0
inf
```

Division by zero yields `inf`/`nan` rather than an error — test with `isfinite`/`isnan` where it matters.

4. Operators and expressions

From loosest to tightest binding:

Level	Operators	Notes
pipe	`|> |>> ~>`	left-assoc; see Functions
logical or	`||`	short-circuit
logical and	`&&`	short-circuit
elementwise or	`|`	on logicals/arrays
elementwise and	`&`	on logicals/arrays
comparison	`== ~= < <= > >=`	elementwise on arrays -> logical array

Chained comparisons. Relational operators chain the way mathematics writes them: `a < b < c` means `a < b` **and** `b < c`, with the middle term evaluated exactly once. Chains must run in one direction — `{<, <=}`, `{>, >=}`, or all `==`; mixing directions is a parse error (write `(a < b) & (b > c)` for that), and `!=` never chains, because `a != b != c` would not mean "all distinct". The conjunction is the elementwise `&`, so a chain over an array is a mask — counting draws in a band is one `sum`:

```
cozy> 0 <= 0.5 < 1
true
cozy> let z = [-1, 0.5, 0.8, 3]; sum(0 < z < 1)
2
cozy> rng(1); sum(-1.96 < randn(1, 100) < 1.96) >= 90
true
```

range | `a:b`, `a:step:b` | inclusive; float steps fine |
additive | `+ -` | |
multiplicative | `* / .* ./ \ .\` | `*` is the **matrix** product on matrices; `.*` elementwise; `\` left division (solve) |
unary | `- ! ~` | binds looser than `^`: `-2^2` is `-4` |
power | `^ .^` | right-assoc: `2^3^2` is `512`; `^` on a square matrix is the matrix power |
postfix | `' .' f(x) a[i] .field` | `'` is **conjugate** transpose, `.'` plain |

The `.`-prefixed operators are the elementwise family, exactly as in Octave:

```
cozy> [1, 2, 3] .* [4, 5, 6]
[4, 10, 18]
cozy> 2 .^ [1, 2, 3]
[2, 4, 8]
```

```
cozy> [1i, 2]'          # conjugate transpose
[  -1i
  2+0i ]
```

Scalars broadcast against arrays in every elementwise operation (`[1, 2] + 1` is `[2, 3]`); two arrays must agree in shape exactly.

5. Variables and scope

`let name = value` is the binding **statement**: at the top level it creates a global; inside a loop body or block it creates a local scoped to that construct. A bare `name = value` (no `let`) is **assignment**: it walks outward through the enclosing scopes and updates the nearest existing `name` — this is how a loop body updates an accumulator defined outside it.

`let name = value in body` is the binding **expression**: `name` is visible only in `body`, and the whole thing evaluates to `body`. Bindings nest and shadow:

```
cozy> let a = 1 in a + (let a = 100 in a) + a
102
```

6. Control flow

`if` is an expression:

```
cozy> let x = 5; if x > 3 then "big" else "small" end
"big"
```

The `else` branch is optional; if the condition is false and there is no `else`, the result is `null`. Conditions must be `Bool` — a number is not a condition.

Loops are statements (they yield `null`):

```
cozy> let s = 0; for i = 1:10 do s = s + i end; s
55
cozy> let n = 1; while n < 100 do n = n * 2 end; n
128
```

`break` leaves the nearest loop, `continue` skips to its next iteration, and `return [value]` exits the enclosing function. All three are safe anywhere — including mid-expression (`acc + (if bad then continue else v end)`): the VM restores the loop's stack state on a non-local exit.

Block expressions sequence statements inside parentheses; the value is the final expression, and `let` bindings are local to the block:

```
cozy> (let x = 3; let y = 4; sqrt(x*x + y*y))
5
cozy> (let q = 7; q); q
error: undefined name 'q'          % block locals do not leak
```

This is the natural shape for a multi-step function body.

`elseif` (since 2.19)

Chains share a single closing `end`; each condition is tried in order:

```
cozy> if 1 > 2 then "a" elseif 3 > 2 then "c" else "d" end
"c"
cozy> let sign_ = fn x -> if x < 0 then -1 elseif x == 0 then 0 else 1 end
<fn/1>
cozy> [sign_(-7), sign_(0), sign_(4)]
[-1, 0, 1]
```

7. Functions

`fn params -> expr` is a lambda; its body is one expression (use a block expression or `let … in` for multiple steps). Functions are values: bind them, pass them, return them.

```
cozy> let f = fn x -> x ^ 2 + 1; f(3)
10
cozy> let add = fn a -> fn b -> a + b; add(2)(5)   # currying
7
cozy> let g = fn x -> (let s = x * x; s + 1); g(4)
17
```

Closures capture by value at creation. Recursion works through the function's own name.

Sections. A parenthesised expression containing `_` becomes a lambda; each `_` is a fresh parameter, left to right: `(_ + 1)` is `fn x -> x + 1`, `(_ * _)` takes two arguments.

map. `map(f, A)` applies `f` elementwise: `map((_ * 10), [1, 2, 3])` is `[10, 20, 30]`.

The pipe. `x |> rhs` feeds a value forward. If `rhs` mentions `@`, the pipe binds `@` to `x` and evaluates `rhs`; if `rhs` is a bare callable, the pipe applies it:

```
cozy> [1, 2, 3, 4] |> sum(@) |> sqrt
3.16228
cozy> 5 |> @ + 1
6
cozy> 9 |> sqrt              # bare callable: sqrt(9)
3
```

Index-bound reductions. Sigma notation, executable: `f[k = R] E` applies any callable `f` to the body `E` evaluated at each `k` in `R` — sugar for `R ~> (fn k -> E) |> f`, so `sum`, `prod`, `max`, `mean`, or your own function all reduce. The binder is scoped to the body; the body binds loose, exactly like a `fn` body, so `sum[k = 1:3] k + 1` sums `k + 1` per term — parenthesize the reduction to operate on its result, and likewise to qualify the range with a `where`: `(sum[k = 1:n] k ^ 2) where n = 3`.

```
cozy> sum[k = 1:100] 1 / k ^ 2
1.63498
cozy> pi ^ 2 / 6
1.64493
cozy> let q = [0.1, 0.2, 0.15]; prod[j = 1:3] (1 - q[j])
0.612
cozy> sum[i = 1:4] sum[j = 1:4] (i == j)
4
```

Where clauses. Any expression can name its constants after the fact, the way mathematics writes them: `expr where a = 1, b = -3`. Bindings are sequential (later ones may use earlier ones — not vice versa), scoped to that one expression (they never leak, and they shadow without damaging outer names), and the clause binds looser than everything else in the expression, so a whole pipeline can be qualified at its end. It is pure sugar for a `let..in` chain. The former `where(...)` builtin is now `find(mask)` (indices of true) and `pick(mask, a, b)` (elementwise select).

```
cozy> let y = a * x ^ 2 + b * x + c where a = 1, b = -3, c = 2, x = 4
6
cozy> sqrt(h) where hs = 3, h = hs + 1
2
cozy> 1:n ~> (@ ^ 2) |> sum where n = 5
55
cozy> b
```

Pipes chain left to right, which reads as a data-flow pipeline.

The elementwise pipe `~>`. Where `|>` feeds the *whole* value, `~>` feeds each *element*: `x ~> f` is `map(f, x)`. The same right-hand-side rules apply, with one deliberate twist: under `~>`, `@` binds the **element**, not the whole array. The operator extends the whole-vs-elementwise distinction (`*` vs `.*`) to pipelines. (In Neutrino, Cozy's ancestor, it was said the elementwise pipe oscillates; the pun retires with the name — the operator does not.) `~>` always means the map primitive itself, so shadowing the name `map` cannot change what the operator does.

```
cozy> [0.5, 1.5, 2.5] ~> (@ * 2) ~> floor
[1, 3, 5]
cozy> 1:5 ~> (@ ^ 2) |> sum
55
cozy> [1, 2, 3] ~> (fn x -> x * 10)
[10, 20, 30]
```

The tee pipe `|>>`. Exactly `|>`, but the value flowing through is printed (echo style) before being passed on — pipeline debugging without dismantling the pipeline. Swap `|>` for `|>>` at the stage you want to watch, then swap back:

```
cozy> [3, 1, 4, 1, 5] |>> sort |>> unique |> length
[3, 1, 4, 1, 5]
[1, 1, 3, 4, 5]
4
```

Fan-out. When the right-hand side of `|>` (or `|>>`) is a record literal, each field's callable is applied to the piped value, and the result is a record of results with the same keys — a `describe()` composed from syntax:

```
cozy> [2.1, 3.7, 1.4, 5.0] |> {n = length, mu = mean, sd = std, top = max}
{n = 4, mu = 3.05, sd = 1.61761, top = 5}
```

Fan-out is one level deep and whole-value only (`~>` into a record is an error). A non-callable field is a type error, and `@` inside the fan-out record is rejected: each field already receives the piped value as its argument.

Default parameter values

Trailing parameters may carry defaults — `fn x, tol = 1e-8 -> ...` — evaluated at call time, left to right, inside the function's own scope, so a later default may reference any earlier parameter. An absent argument takes its default; so does an explicit `null`, which lets a call skip a middle parameter while supplying a later one. A function with defaults reports its arity as a range (`<fn/1..3>`), and calling outside the range names the range in the error.

```
cozy> let bisect = fn f, a, b, tol = 1e-10 -> fzero(f, a, b)
<fn/3..4>
cozy> let root = fn f, a, b, tol = 1e-10, verbose = false -> fzero(f, a, b)
<fn/3..5>
cozy> root
<fn/3..5>
cozy> let g = fn a, b = a * 2, c = a + b -> [a; b; c]
<fn/1..3>
cozy> g(10)'
[10, 20, 30]
cozy> g(10, null, 99)'
[10, 20, 99]
```

Help for your own functions

`help(f)` on a function you or a package defined shows its signature (taken from the code, defaults included), the comment block written directly above its `let` as the description, and that block's `e.g.` lines as examples — the same shape builtin help has. `help("name")` asks by name (the way to read about a zero-argument function without autocalling it), and `help("pkg")` lists a loaded package's functions with their first line. The package guide states the docstring law and how the suite verifies every package example.

```
cozy> let sq = fn x, p = 2 -> x^p
<fn/1..2>
cozy> help(sq)
  sq(x, p = 2)
      your function, 1 to 2 arguments
```

Write a comment block directly above a `let` in a file you load and it becomes the description; an `e.g.` line becomes the example (`data/cube.cz` ships as the smallest demonstration):

```
% cube(x): x to the third power.
% e.g. cube(2)   %= 8
let cube = fn x -> x^3
```

```
cozy> load("data/cube.cz")
cozy> help(cube)
  cube(x)
      x to the third power.
      package function, 1 argument  (data/cube.cz)
      e.g.  cube(2)   % 8
```

When something goes wrong

A runtime error names the whole call chain, innermost frame first — captured at the moment of the error, so errors raised inside closures that optimizers or `map` re-enter keep their trace:

```
cozy> let inner = fn x -> x[5]
cozy> let middle = fn v -> inner(v) * 2
cozy> middle([1; 2])
error:  index 5 out of bounds (1..2)
    in inner (line 1)
    in middle (line 1)
```

Line numbers are relative to where each function was defined (its file line under `load`, its own line at the prompt). Deep recursion shows the first eight frames and counts the rest.

8. Arrays and matrices

Matrix literals use `,` between columns and `;` between rows; every array is two-dimensional and **1-indexed**. `[]` is the empty array.

Indexing supports scalars, ranges, `:` (everything along a dimension), `end` (the last index, with arithmetic), index vectors (gather), and logical masks:

```
cozy> let A = [1, 2, 3; 4, 5, 6]
[ 1  2  3
  4  5  6 ]
cozy> size(A)
[2, 3]
cozy> A[2, 3]          # element
6
cozy> A[1, :]          # row
[1, 2, 3]
cozy> A[:, 2]          # column
[ 2
  5 ]
cozy> A[end, end]
6
cozy> let v = [10, 20, 30, 40]; v[2:3]
[20, 30]
```

```
cozy> v[v > 15]        # logical mask
[20, 30, 40]
cozy> v[[1, 4]]        # gather
[10, 40]
```

Indexed assignment works with the same selectors, copy-on-write: `A[1, 2] = 9`, `v[v < 0] = 0`, `A[2, :] = [7, 8, 9]`. The target must be a plain name and indices must be in range (no auto-growing).

`unique(A)` returns the sorted distinct elements — vectors keep their orientation, matrices flatten to a row. **Logical arrays** come from comparisons and drive masking and counting: `sum(A > 2)` counts, `any`/`all` test, `find(mask)` gives 1-based positions, `pick(mask, a, b)` selects elementwise.

Construction and reshaping: `zeros`, `ones`, `eye`, `diag`, `linspace`, `reshape` (row-major), `repmat`, ranges `1:n`. **Reductions** take an optional dimension: `sum(A, 1)` down columns, `sum(A, 2)` across rows, `max(A, [], dim)` for the extrema.

Descriptive statistics follow the same conventions. `var` and `std` normalize by N-1 by default (`var(A, 1)` divides by N), `median` handles even counts by averaging, and `quantile` uses linear interpolation between order statistics (the NumPy default):

```
cozy> let x = [2, 7, 4, 9, 3]
cozy> var(x)
8.5
cozy> quantile(x, [0.25, 0.5, 0.75])
[3, 4, 7]
```

`cov(X)` and `corr(X)` treat a matrix's **columns as variables** and rows as observations, returning the p x p covariance / Pearson correlation matrix (`cov` takes the same `w` normalization as `var`). On two vectors they return the scalar: `cov(x, y)`, `corr(x, y)`. A constant column has no correlation to speak of — those entries are `nan`:

```
cozy> corr([1, 2, 3, 4, 5], [2, 1, 4, 3, 5])
0.8
```

9. Linear algebra

`*` is the matrix product; `\` solves. Square systems use LU with partial pivoting; non-square `\` is least squares — overdetermined gives the LS fit, underdetermined the minimum-norm solution.

```
cozy> [2, 1; 1, 3] \ [3; 5]
[ 0.8
  1.4 ]
cozy> [1, 1; 1, 2; 1, 3] \ [1; 2; 2]     # regression: intercept, slope
[ 0.666667
       0.5 ]
```

The decompositions return records, so the pieces have names:

Call	Returns	Identity
`lu(A)`	`{L, U, p}`	`P*A = L*U`
`qr(A)`	`{Q, R}`	`A = Q*R`
`chol(A)`	`L`	`L*L' = A` (Hermitian PD)
`svd(A)`	`{U, S, V}`	`A = U*diag(S)*V'`
`eig(A)`	`{values, vectors}`	`A*V = V*diag(values)`

`eig` handles both Hermitian matrices (Jacobi; real ascending eigenvalues, orthonormal vectors) and general ones (complex QR + inverse iteration; complex pairs come out right). All of it is complex-capable — `'` being

the conjugate transpose is what makes the identities hold. `kron(A, B)` is the Kronecker product, `det`, `inv`, `trace`, `norm`, `dot` round out the set.

10. Complex numbers

The imaginary literal is a number followed by `i`. Complex values propagate through arithmetic and the math library; results stay complex (`1i * 1i` is `-1+0i`, not `-1`). `real`, `imag`, `conj`, `angle` access the parts, `abs` is the modulus. Functions with limited real domains return complex off it: `sqrt(-4)` is `2i`, `log(-1)` is `3.14159i`, `asin(2)` is complex.

10b. Dual numbers and exact derivatives

A dual number is `a + b*eps` with `eps^2 = 0` — the derivative-carrying sibling of complex. `dual(a, b)` builds one (elementwise over arrays), `dualval`/`dualeps` read the parts, and both accessors are total on plain numbers (`dualval(7)` is `7`, `dualeps(7)` is `0`), so a function whose branch returns a constant still differentiates. Arithmetic and the whole transcendental library apply the chain rule exactly: no step size, no differencing. Comparisons read the value part, so conditionals inside a differentiated function take the branch the values take (the derivative of `abs` at its kink is one-sided). Dual and complex do not mix — that is the promotion law, and mixing them is an error naming `dualval`. The dense linear-algebra kernels (`eig`, `svd`, `\`, `det`, `norm`) gate on dual matrices; autodiff flows through elementwise ops, `*`, and reductions, which is what objective functions are made of.

```
cozy> dual(3, 1) * dual(3, 1)
9+6eps
cozy> sin(dual(0, 1))
1eps
cozy> dualeps(sqrt(dual(4, 1)))
0.25
```

Hyper-duals carry second derivatives: `hdual(x, s1, s2)` seeds two directions at once (`eps1^2 = eps2^2 = 0`, their product survives), and `hdual12` reads the exact mixed partial from one pass — `hdual12(hdual(3, 1, 1)^2)` is exactly `2`. The same promotion law applies twice over: hyper-dual mixes with neither complex nor dual. `gamma`/`lgamma` refuse hyper-duals (their second derivative needs trigamma — a recorded residue).

`load("packages/autodiff.cz")` turns all of this into `d(f)`, `grad(f)`, and `hess(f)` — see the packages guide.

11. Special functions and statistics

The special-function set is chosen as *primitives* from which the classical distributions are one-liners:

```
cozy> let normcdf = fn x -> 0.5 * erfc(-x / sqrt(2))
cozy> normcdf(1.96)
0.975002
cozy> norminv(0.975)
1.95996
```

`gammainc(x, a)` is the regularized lower incomplete gamma — the chi-square CDF is `gammainc(x/2, k/2)`. `betainc(x, a, b)` is the regularized incomplete beta — Student-t and F CDFs fall out of it. `erf`/`erfc`, `beta`/`lbeta`, `gamma`/`lgamma`, `digamma`, and integer-order Bessel `besselj`/`bessely` complete the set. All are real-domain, elementwise over arrays, and validated against SciPy.

Root finding, minimization, integration

These three take a **function argument** — a closure or builtin — and call it repeatedly. `fzero(f, a, b)` finds a root by Brent's method (`f(a)` and `f(b)` must differ in sign); `fminbnd(f, a, b)` minimizes on an interval and returns `{x, fx}`; `integral(f, a, b[, tol])` integrates adaptively (Simpson with Richardson estimate;

finite limits; default tolerance 1e-10). Closures capture data, so parameterized problems read naturally — a bond's yield to maturity:

```
cozy> let cf = [100, 100, 100, 1100]
cozy> let npv = fn r -> sum(cf ./ ((1 + r) .^ (1:4))) - 1000
cozy> format(6); fzero(npv, 0.01, 0.3)
0.100000
```

A divergent or wildly oscillatory integrand fails with an error rather than a wrong answer; infinite limits are rejected — substitute to a finite domain first.

Strings

Strings are byte sequences (UTF-8 passes through but is not interpreted; `length` and indexing count bytes). `+` concatenates; `==`, `!=`, `<` and friends compare lexicographically, shorter prefixes first; indexing uses the same machinery as arrays, so ranges, `end`, and even permutations work:

```
cozy> let s = "cozy"
"cozy"
cozy> s[1:3] + "!" + s[end]
"coz!y"
cozy> "apple" < "banana"
true
```

The library: `upper`, `lower`, `trim`, `contains(s, sub)`, `startswith(s, p)`, `endswith(s, p)`, `strrep(s, old, new)`. Two bridges: `str(x)` gives any value's display text as a string, `num(s)` parses a string as a number (Int if exact, else Float). And `fmt(tmpl, ...)` is `print`'s template engine returning a string instead of printing:

```
cozy> fmt("run {} done in {:.2f}s", 3, 1.234)
"run 3 done in 1.23s"
```

Strings also form **arrays**: `["a", "bb"; "c", "dd"]` is a 2x2 String matrix (mixing strings with numbers in one matrix is an error). Indexing, slicing, `end`, transpose, `reshape`, `sort`, and `unique` all work, and elementwise operations do what a data-frame user hopes:

```
cozy> let names = ["si", "at", "de", "si"]
cozy> names == "si"
[true, false, false, true]
cozy> names[names == "si"]
["si", "si"]
cozy> ["pre_", "un_"] + "fix"
["pre_fix", "un_fix"]
```

Numeric reductions (`min`, `max`, `sum`, `norm`, …) refuse string arrays rather than computing nonsense, and assignment cannot mix kinds: a String cell never silently becomes a number, nor the reverse.

`strsplit(s, sep)` and `strjoin(a, sep)` convert between strings and string arrays; `fields(r)` returns a record's field names as a string column; `readtable` loads non-numeric CSV columns as string arrays (quoted cells, RFC-4180 style, are handled); `writecsv` writes string matrices with proper quoting; and plots take `{labels = ["low", "high"]}` for multi-series legends alongside the older `label1`, `label2`, … form.

String-and-number arithmetic is still an error — there is no implicit conversion in either direction; use `str` and `num` to cross the bridge.

Where, not just whether: `strfind` — and dynamic record fields

`contains` says whether a pattern occurs; `strfind(s, pat)` says where — every 1-based start position (overlapping occurrences count), `[]` if none. Since strings index like arrays, pattern-directed slicing is now a one-liner instead of a character scan. And the record reflection pair closes the mirror-image gap: `getfield(r, name)` reads a field named at runtime (strict error if absent, like literal access), and `setfield(r, name, v)` returns

a *new* record with the field replaced or appended — records stay immutable values, and construction is a fold: grow `{}` one `setfield` at a time. Generic column statistics, record merge, k=v parsing, and serialization round-trips now live in packages.

```
cozy> let s = "key=value"; s[strfind(s, "=")[1]+1 : end]
"value"
cozy> strfind("mississippi", "ss")
[3; 6]
cozy> let t = {yr = [2024; 2025], cpi = [3.1; 2.4]}; fields(t) ~> (fn nm -> mean(getfield(t, nm)))
[2024.5; 2.75]
cozy> fields({} |> setfield(@, "lo", 1) |> setfield(@, "hi", 2))
["lo"; "hi"]
```

Packages: `load`

`load("file.cz")` runs a file in the current session; its `let` bindings persist afterwards. A package is just a file of definitions — and a record of closures makes a namespace, so packages don't collide. Namespaces need no new machinery: `fields(r)` is the manifest, `getfield(r, name)` the dynamic door, and sibling fields may call each other through the record's own global name (`stats.z` calling `stats.se` works, because functions resolve globals at call time). The same late binding is the one law worth memorizing: **a record namespace hides the face, never the body** — a field's body still resolves its helpers as globals at call time, so `keep()` of the record alone strands every call on `undefined name`. The standard packages therefore tag-prefix their helpers (`op_`, `ad_`, `sl_`); the full authoring convention is in the packages guide under "Writing your own". Example:

```
cozy> load("tests/data/mathlib.cz")
cozy> cube(4)
64
cozy> geo.hyp([3, 4])
5
```

`save("ws.cz")` writes the whole workspace — every variable and function — as reloadable source; restore it with `load("ws.cz")`. The file is plain Cozy, so it is readable and editable. Functions that capture variables (closures made by other functions) cannot be serialized and refuse with a clear message; a failed save leaves no file behind. `body(f)` prints a function's retained source, and `ast(f)` goes one further — the body as a walkable record tree ({op, l, r, …}, symb-compatible), which is how the book's Taylor problem differentiates `sin` symbolically seven times. `body(f)` prints the source of a user-defined function:

```
cozy> let cube = fn x -> x^3
cozy> body(cube)
fn x -> x^3
```

Expressions may span lines wherever a `(`, `[`, or `{` is open — newlines there are plain whitespace (matrix rows still take an explicit `;`). At the top level a newline ends the statement, and the REPL reads continuation lines automatically while a bracket is open.

Packages validate their inputs with `error` and `assert`:

```
cozy> let f = fn x -> (assert(x > 0, "f needs x > 0, got {}", x); sqrt(x))
cozy> f(-4)
error: f needs x > 0, got -4
```

The standard packages — distributions, polynomials, finance, and the solar almanac — are documented with verified examples in PACKAGES.md. Packages can `load` other packages (nesting is capped, so circular loads error cleanly). Closures capture by value, so packages are libraries of functions rather than stateful modules. Errors inside a loaded file are reported with the file name; a parse error includes its line and column within the file.

Masks and reductions

Comparisons on arrays yield Bool masks, and the reductions treat `true` as 1: `sum` of a mask counts, `mean` of a mask is the fraction — the one-line statistic for any condition.

```
cozy> let mask = [1, 5, 2, 8, 3] > 2.5
[false, true, false, true, true]
cozy> sum(mask)
3
cozy> mean(mask)
0.6
```

12. Random numbers

The generator is xoshiro256** seeded through splitmix64, and it is **reproducible by default**: every fresh session starts from the same fixed seed, so a script's random draws are stable run to run. `rng(seed)` reseeds — same seed, same stream. `rand`, `randn`, `randi` draw uniform, normal, and integer variates, scalar or matrix-shaped (`rand(3)`, `randn(2, 4)`).

13. Plotting

Plotting has three backends. **Natively** the default is **gnuplot**, out of process — a soft dependency: the language works without it, and `plot` reports cleanly if it is missing (`plot: gnuplot failed (exit 127) - is gnuplot installed?`). Setting `COZY_PLOT_TERM=ascii` renders deterministic text plots into the terminal instead, and `COZY_PLOT_TERM=svg` writes standalone `plot_N.svg` files (dark palette). In the **browser** the default is the SVG backend, rendered into the workbench's Plots pane. For scatter plots, `packages/scatter.cz` wraps the `style = "points"` path every backend honors.

`plot(y)` plots a vector against its index; `plot(x, y)` plots pairs; if `y` is a **matrix**, each column is a separate series. An optional trailing argument is either a gnuplot style string (`"points"`, `"lines lw 2"`, `"impulses"`) or an options record. The svg and ascii backends honor the marker family by substring — any style mentioning `point`, `circle`, or `dot` draws markers, everything else draws lines — so `"circle"` means the same thing on every backend:

```
cozy> let x = linspace(0, 10, 200)
cozy> plot(x, map(sin, x), {title = "sin(x)", xlabel = "x", grid = true})
```

Recognised options: `title`, `xlabel`, `ylabel`, `style` (strings); `logx`, `logy`, `grid` (booleans); `xrange`, `yrange` (`[lo, hi]` vectors) to fix an axis instead of letting gnuplot choose; and legend labels — `label` for a single series, `label1`, `label2`, … for several (unlabeled series keep the `series k` default):

```
cozy> let t = (linspace(0, 6.28, 100))';
cozy> plot(t, [map(sin, t), map(cos, t)], {label1 = "sin", label2 = "cos"})
``` `hist(y)` draws a histogram
(`hist(y, nbins)` to choose the bin count; the default follows Sturges' rule),
and accepts the same trailing options record:
```

cozy> rng(7) cozy> hist(randn(1, 5000), 40)

```
A word of caution that `yrange` exists to address: gnuplot auto-ranges the
y-axis to the data, which can make pure sampling noise look like structure. A
histogram of 100 000 uniform draws in 20 bins has bin counts of 5000 +/- 69
(one binomial standard deviation) - auto-ranged, that +/-1.4% wiggle fills the
whole plot and looks alarming; anchored at zero it is the flat wall it should
be:
```

cozy> hist(rand(1, 100000), 20, {yrange = [0, 6000]})
```

Plots open in a gnuplot window that outlives the command (`gnuplot -persist`).
For scripted rendering, two environment variables redirect output -
`COZY_PLOT_TERM` sets the gnuplot terminal and `COZY_PLOT_OUT` the
file:

```sh
COZY_PLOT_TERM="pngcairo size 800,500" COZY_PLOT_OUT=fig.png \
  ./cozy script.cz
```

(`COZY_PLOT_TERM="dumb size 76,20"` draws ASCII plots straight into the terminal, which is occasionally exactly what you want.) Complex data is rejected — plot `real(z)`, `imag(z)`, or `abs(z)` explicitly.

14. Data files

`readcsv(file)` reads a numeric CSV into a Float matrix; `writecsv(file, A)` writes one at full precision, so values **round-trip bit-exactly**. Empty cells become `nan` (missing data), Windows line endings are tolerated, and `{delim = ";", skip = n}` options handle other separators and preamble lines. Ragged rows and non-numeric cells are errors that name the row and column.

`readtable(file)` reads a CSV whose first line is a header and returns a **record of column vectors**, keys sanitized from the column names (`"GDP Growth (%)"` becomes `gdp_growth`; duplicates get `_2`, `_3`, …). This is Cozy's data frame — named columns plus the existing mask machinery:

```
cozy> writecsv("/tmp/m.csv", [1, 2; 3, 4]); readcsv("/tmp/m.csv")
[ 1  2
  3  4 ]
```

```
cozy> let d = readtable("tests/data/macro.csv")
cozy> mean(d.gdp_growth)
1.93333
cozy> d.cpi[d.year >= 2021]
[  nan
     8 ]
```

For a headerless file use `readcsv` — `readtable` will take the first line as a header regardless of its content. A column containing text (country names, tickers) is rejected with an error naming the column: representing it needs first-class strings, which the language does not yet have.

Where load looks

A bare name — `load("dist")` — searches, in order: the literal path, `packages/NAME` and `packages/NAME.cz`, `NAME.cz` in the current directory, each colon-separated directory in `$COZY_PATH`, and finally beside the running binary (so a symlinked `cozy` finds its own packages from anywhere). Names already ending `.cz` — exactly what `ls("packages")` hands out — are never double-suffixed, and a miss names every place searched. `ls` with a bare directory name falls back beside the binary the same way.

15. Output and formatting

Number display has one rule worth internalizing: **`format(n)` sets significant digits** (printf `%g`), which is why `format(2)` shows 100.51 as `1.0e+02` — two significant digits genuinely cannot spell 100.51. When you want *decimals* — bills, prices, fixed-width tables — say so explicitly with the systematic two-argument form:

- `format("fixed", d)` — d digits after the point (`%f`): the bill mode.
- `format("sci", d)` — scientific with d decimals (`%e`).
- `format("auto", d)` — d significant digits (`%g`); `format(d)` is its shorthand.

- `format` with no arguments reports the current setting; `format("default")` restores the terse startup style. The Octave-style names (`"short"`, `"long"`, `"short f"`, `"long f"`, `"short e"`, `"long e"`) remain as presets.

```
cozy> let bill = 87.40; bill * 1.15
100.51
cozy> format(2)
cozy> bill * 1.15
1.0e+02
cozy> format("fixed", 2)
cozy> bill * 1.15
100.51
cozy> ans / 4
25.13
cozy> format("sci", 3)
cozy> ans
2.513e+01
cozy> format
format: scientific, 3 decimals
cozy> format("default")
```

Strings interpolate with `fmt`, and `print` shares its template semantics (`{}` placeholders; double the braces for literals).

15b. Strings as code, and the keyboard (since 2.19)

`eval` runs a string as Cozy code in the current session and returns its last value — which, among other things, gives dynamic record access: `eval("w." + col)`. `names` is the programmatic sibling of the `who` family (whose printed tables are unchanged): your workspace names as a sorted string column, with `"vars"`/`"funcs"` selectors.

```
cozy> eval("2 + 2")
4
cozy> let w = {temp = [21.5; 19.8], rain = [0; 4.2]};
cozy> let col = "temp"; eval("w." + col)
[21.5; 19.8]
cozy> let a = 1; let f = fn x -> x; names("vars")
["a"; "ans"; "col"; "w"]
cozy> names("funcs")
["f"]
```

`input("prompt")` reads one line from the keyboard as a string, and `pause()` waits for Enter — `window.prompt` and an alert in the browser. Interactive by nature, so shown here rather than machine-replayed (under a pipe they consume the next stdin line; `tests/run_io.sh` checks exactly that):

```
let name = input("who are you? ");
print("hello, " + name)
pause("press Enter for the plot...")
```

Binding builtins: functions are values, commands autocall

`version` is a function; `let v = version` stores *the function* (like `let s = sin`), which `whof` lists. A bare zero-argument builtin — including through an alias — autocalls at the top level and prints its result; in argument position it does not, so `length(v)` measures the function value (1). To store the *result*, call it: `let w = version()` — then `w` is an ordinary string variable. (The version string itself is suppressed below with `;` so this page never goes stale.)

```
cozy> let v = version
```

```
<builtin version>
cozy> whof
  v            builtin
cozy> length(v)
1
cozy> let w = version();
cozy> w == version()
true
cozy> names("vars")
["ans"; "w"]
```

Which kernels answered: buildinfo

`buildinfo()` returns a record identifying the build: `backend` names the linear-algebra kernel table linked into this binary (`"tier0"` is the zero-dependency hand-rolled set; accelerated backends are a build-time choice — `make BACKEND=...` — never a language-visible one), `version` matches `version()`, and `built` embeds the compile date and time. A production tool whose user cannot tell an accelerated backend from the fallback kernels is not verifiable; this is the introspection that makes the difference visible. Build with `make BACKEND=openblas` for LAPACK-backed kernels (`eig(rand(300))` measured ~158x faster than the hand-rolled tier0); `make` alone keeps the zero-dependency tier0. Either way the language is identical — the conformance suite passes byte-for-byte under both.

Optimization has the same shape of choice, orthogonal to the linear algebra: `make OPTIM=nlopt` (Debian/Ubuntu: `libnlopt-dev`; macOS: `brew install nlopt`) links NLopt and enables the `nlmin` builtin — SLSQP, L-BFGS, BOBYQA, and COBYLA behind one options record, with gradients and constraint Jacobians supplied EXACTLY by Cozy's dual numbers rather than finite differences. `minimize_con` and `maximize_con` in the optim package dispatch to SLSQP automatically when `buildinfo().optim` says `"nlopt"`; without the backend they run the pure augmented-Lagrangian path. The browser build carries NLopt too — a vendored C subset compiled into the bundle — so the tab optimizes with the same algorithms as the desktop. `minimize_newton` never dispatches: exact hyper-dual Newton is native Cozy either way.

The optimizers also have a binder surface, the index-bound notation generalized from sigma: `minimize[x = x0] f(x)` reads as mathematics and desugars to `minimize(fn x -> f(x), x0)` — likewise `maximize` and `nlmin`. For every other callable, `f[k = R] E` keeps its reduction meaning; the optimizer names are special-cased because a minimization binds a starting point, not a range. (The transcript below shows only the record's shape: the backend name varies by build and the timestamp by the minute, so this page never goes stale.)

```
cozy> fields(buildinfo())
["backend"; "version"; "built"; "optim"]
cozy> let b = buildinfo();
cozy> b.version == version()
true
```

Sparse matrices

A sparse matrix is its own kind of value — CSR storage, float or complex — built by `sparse(A)` from a dense matrix or `sparse(i, j, v, m, n)` from 1-based triplets (duplicates summed, zeros never stored), plus `speye(n)`, `sprand(m, n, d)`, and `sprandn(m, n, d)` (both drawing from the session's reproducible RNG). `dense(S)` crosses back; `nnz` counts stored entries; `who` shows `sparse RxC, nnz = N`. The promotion law is stated once: zero-preserving operations stay sparse (`S + S`, `S .* S`, `k * S`, `-S`, `S'`), the founding kernel is sparse-matrix × dense-column (`S * v`), and anything that would silently densify — `S + 1`, `S == S`, `S \ b` — is an error that names the way through (`dense(S)` if you meant it). Iterative solvers built on `S * v` are the intended path for large systems. Indexing reads are scalar for now: `S[i, j]`.

```
cozy> let S = sparse([0, 5; 3, 0]); S
sparse 2x2, nnz = 2
```

```
  (1,2)  5
  (2,1)  3
cozy> S * [10; 100]
[500; 30]
cozy> let T = sparse([1; 2; 2], [1; 1; 2], [4; 7; 9], 3, 3); nnz(T)
3
cozy> nnz(sprand(100, 100, 0.05))
500
```

16. Scripts and tools

`cozy file.cz` runs a script (top level is a statement sequence; `#`/`%` comments). The binary also exposes the compiler pipeline:

Invocation	Shows
`cozy --tokens file.cz`	the token stream
`cozy --ast file.cz`	the parse tree
`cozy --dis file.cz`	the compiled bytecode, statement by statement

`tic` starts a monotonic wall-clock timer and `toc()` returns the elapsed seconds — bare `toc` as a statement echoes it, but in an expression position write `toc()` (a bare name is a value reference, as with any function):

```
cozy> tic; let A = randn(100, 100); let B = A * A; toc() < 60
true
```

`vmtest` is the headless driver used by the test suite: it reads stdin line by line and echoes each result, so `printf 'sum(1:100)\n' | ./vmtest` prints `5050`. `dis(f)` disassembles from inside the language. The golden suite (`make test`) and its ASan twin (`make test-asan`) are how changes prove themselves; `tests/dis/` pins the emitted bytecode for core constructs.

SVG plots. `COZY_PLOT_TERM=svg` makes `plot` and `hist` write `plot_N.svg` files instead of using gnuplot or ASCII. The browser build uses this by default: plots appear in the page's Plots panel, and "download new files" saves them.

Editors

Emacs. `editors/cozy-mode.el` provides a major mode for `.cz` files: syntax highlighting (the builtin list is generated from the interpreter's own documentation table, so it cannot drift), `%` comments, block-aware indentation, and an inferior REPL. Put the file on your `load-path` and `(require 'cozy-mode)`; then `M-x run-cozy` starts the REPL, and from any `.cz` buffer `C-c C-r` sends the region, `C-c C-b` the buffer, `C-c C-l` loads the file, `C-c C-z` jumps to the REPL. On GitHub, a `.gitattributes` rule highlights `.cz` as Octave — close enough until linguist learns Cozy.

The workspace stays readable: load groups

`load` remembers which names each file defined, and `who` collapses every loaded file to one summary line — so after a package-heavy session your own definitions stay findable at a glance. `who("name")` (the file's basename, or its full path) opens a shelf in full; `who("all")` restores the flat listing; the kind filters (`who("functions")`, `who("vars")`, …) remain flat, as before.

```
cozy> load("packages/scatter.cz")
cozy> let zq = 5
5
cozy> who
  packages/scatter.cz      3 names   (who("scatter") to list)
  zq           int        = 5
```

```
cozy> who("scatter")
  scatter      function   (2 params)
  scatter_titled function   (3 params)
  jitter       function   (2 params)
```

Re-loading a file replaces its shelf; `clear`ed names drop from the counts; and `clear("scatter")` unloads a whole shelf — every member removed, the summary line gone, your own names untouched (a variable with the same name as a shelf wins the dispute). The registry follows names, not values — a rebound name stays on its shelf.

```
cozy> load("packages/scatter.cz")
cozy> let zq = 5
5
cozy> clear("scatter"); who
  packages/scatter.cz      2 names   (who("scatter") to list)
  zq           int        = 5
```

17. Builtin reference

Generated from the interpreter's own documentation table (the same data ***`help`*** *shows), so it cannot drift from the implementation.*

Core & introspection

Signature	Description
`print(...) | print(tmpl, ...)`	print values; template fills {} in order; {:[-][w][.p][f|e|g]} formats a hole ({{ }} literal)
`fields(r)`	the record's field names, as a string column
`getfield(r, name)`	dynamic field read; error if the record has no such field
`setfield(r, name, v)`	a new record with the field replaced or appended; r is untouched
`error(msg) | error(tmpl, ...)`	raise a runtime error (fmt-style template)
`assert(cond) | assert(cond, tmpl, ...)`	error unless cond is true
`version`	the interpreter version, as a string
`buildinfo()`	build introspection -> {backend, version, built}; backend names the linear-algebra kernels
`now`	current local date and time: {y, m, d, h, mi, s}
`whov | whov("sorted")`	your variables only (shorthand for who("vars"))
`whof | whof("sorted")`	your functions only (shorthand for who("functions"))
`whor | whor("sorted")`	your records only (shorthand for who("records"))
`whos`	the whole workspace, sorted by name (who("sorted"))
`who | who("functions", "sorted")`	list the workspace; filter by "records"/"functions"/"vars", add "sorted" for name order
`help / help(f)`	help lists every builtin; help(f) describes one
`system(cmd)`	run a shell command string; return its exit status
`dis(f)`	disassemble a function's bytecode (compiler/VM introspection)
`format / format(n) / format(mode, digits)`	number display: format(n) sets SIGNIFICANT digits; format("fixed", d) / format("sci", d) / format("auto", d) set the mode and digits explicitly; format() shows the current setting
`size(x)`	[rows, cols] of x (a scalar is 1x1)

Signature	Description
`length(x)`	longest dimension of x (0 if empty)
`numel(x)`	number of elements (rows*cols)
`save("file.cz")`	write all variables and functions as reloadable source (restore with load)
`body(f)`	print the source of a user-defined function
`load("name" | "file.cz")`	run a file in the current session; bare names search packages/, $COZY_PATH, and beside the binary (a miss names every place tried); let-bindings persist
`eval("code")`	run a string as Cozy code in this session; returns the last value
`names() | names("vars"|"funcs")`	your workspace names as a sorted string column (the programmatic who)
`clear() | clear("a", ...)`	remove all user variables, or the named ones; clearing a shadow restores the standard-library original
`keep("a", "b", ...)`	remove all user variables except the named ones (the complement of clear)
`mem`	print workspace size (variables) and peak process memory
`tic`	start the wall-clock timer (monotonic)
`toc`	seconds elapsed since tic
`ast(f)`	quote a function: its body as a symb-style record tree ({op, l, r, …}); params as a string row
`date("2024-03-15") | date(y, m, d)`	a calendar Date (days since 1970-01-01 UTC, fractional time); strict algebra: Date +- number -> Date, Date - Date -> days, anything else errors
`datestr(d)`	a date rendered "YYYY-MM-DD[HH:MM:SS]" as a string
`today()`	the current UTC date
`year(d)`	calendar year of a date or date column
`month(d)`	calendar month (1-12) of a date or date column
`day(d)`	day of month of a date or date column
`quarter(d)`	calendar quarter (1-4) of a date or date column
`weekday(d)`	day of week, Monday = 1 .. Sunday = 7
`packeur`	interactive package picker: a checklist of the .cz packages beside the session (arrows/jk move, space toggles, Enter applies, q cancels); headless stdin drives it in scripts

Strings

Signature	Description
`upper(s)`	uppercase (ASCII bytes)
`lower(s)`	lowercase (ASCII bytes)
`trim(s)`	strip leading and trailing whitespace
`contains(s, sub)`	true if sub occurs in s
`startswith(s, p)`	true if s begins with p
`endswith(s, p)`	true if s ends with p
`strrep(s, old, new)`	replace every occurrence of old with new
`str(x)`	the display text of any value, as a string
`num(s)`	parse a string as a number (Int if exact, else Float)

Signature	Description
`fmt(tmpl, ...)`	print's template, returned as a string instead of printed
`strsplit(s, sep)`	split a string on a separator, giving a string row vector
`strfind(s, pat)`	1-based start positions of every occurrence of pat in s (overlapping), [] if none
`strjoin(a, sep)`	join a string array with a separator

REPL commands

Signature	Description
`exit | exit(code)`	end the session (also: quit)
`manual [doc]`	page rendered documentation: manual, manual book|packages|changelog|lessons|design|readme
`pretty on|off`	aligned multi-line matrix display (default on in the REPL)
`more on|off`	page long output through $PAGER

Solvers

Signature	Description
`fzero(f, a, b)`	root of f in [a, b] (Brent; f(a), f(b) must differ in sign)
`fminbnd(f, a, b)`	minimum of f on [a, b] (Brent) -> {x, fx}
`integral(f, a, b[, tol])`	definite integral (adaptive Simpson, finite limits; default tol 1e-10)

Data files

Signature	Description
`readcsv(file[, opts])`	numeric CSV -> Float matrix; empty cells are nan; opts: {delim, skip}
`writecsv(file, A[, opts])`	matrix -> CSV, full precision (round-trips); opts: {delim}
`readtable(file[, opts])`	CSV with a header -> record of column vectors named from the header
`pwd`	the current working directory, as a string
`cd("dir") | cd`	change the working directory (persists, unlike !cd); bare cd goes home
`ls | ls("dir") | ls("*.cz")`	directory listing as a string array (globs supported)
`input("prompt")`	read one line from the keyboard as a string (window.prompt in the browser)
`pause() | pause("msg")`	wait for the user before continuing (alert in the browser)

Plotting

Signature	Description
`plot(y) | plot(x, y) | plot(x, Y, opts)`	line plot via gnuplot; Y columns are series; opts: style string or {title, xlabel, ylabel, style, logx, logy, grid, xrange, yrange, label, label1..labelN}
`hist(y[, nbins][, opts])`	histogram via gnuplot; opts as in plot (yrange to anchor the axis, label for the legend)

sparse

Signature	Description
`sparse(A) / sparse(i, j, v, m, n)`	a sparse (CSR) matrix from a dense one, or from 1-based triplets (duplicates summed)
`dense(S)`	the dense matrix a sparse one represents (the explicit gate in the promotion law)
`nnz(A)`	the number of stored nonzeros (sparse) or nonzero entries (dense)
`speye(n)`	the n-by-n sparse identity
`sprand(m, n, d)`	a sparse m-by-n matrix with ~dmn uniform(0,1) entries at distinct random positions
`sprandn(m, n, d)`	like sprand with standard-normal values

Array construction

Signature	Description
`zeros(r, c)`	r-by-c matrix of zeros
`ones(r, c)`	r-by-c matrix of ones
`eye(n)`	n-by-n identity matrix
`diag(x)`	vector -> diagonal matrix; matrix -> its diagonal as a column
`linspace(a, b, n)`	row of n points evenly spaced from a to b inclusive
`reshape(A, r, c)`	reinterpret A's elements as r-by-c (row-major), element count must match
`repmat(A, m, n)`	tile A into an m-by-n grid of copies

Reductions

Signature	Description
`sum(A) | sum(A, dim)`	sum of all elements, or along dim (1 = columns, 2 = rows)
`prod(A) | prod(A, dim)`	product of all elements, or along dim
`cov(X[, w]) | cov(x, y[, w])`	covariance matrix of X's columns (rows = observations), or scalar cov of two vectors; w as in var
`corr(X) | corr(x, y)`	Pearson correlation matrix of X's columns, or scalar correlation of two vectors
`var(A) | var(A, w) | var(A, w, dim)`	variance; w = 0 divides by N-1 (default), w = 1 by N
`std(A) | std(A, w) | std(A, w, dim)`	standard deviation (sqrt of var, same normalization)
`median(A) | median(A, dim)`	median of all elements, or along dim

Signature	Description
`quantile(x, p)`	quantile(s) of the data at probability p (scalar or vector); linear interpolation
`mean(A) | mean(A, dim)`	mean of all elements, or along dim
`min(A) | min(a, b) | min(A, [], dim)`	smallest element; elementwise min; or min along dim
`max(A) | max(a, b) | max(A, [], dim)`	largest element; elementwise max; or max along dim
`any(mask) | any(mask, dim)`	true if any element is nonzero/true (overall or along dim)
`all(mask) | all(mask, dim)`	true if every element is nonzero/true (overall or along dim)

Constants

Signature	Description
`pi`	3.14159…, the circle constant
`e`	2.71828…, Euler's number
`eulergamma`	0.57722…, the Euler-Mascheroni constant
`phi`	1.61803…, the golden ratio
`eps`	machine epsilon for Float (2^-52)
`inf`	positive infinity (Float)
`nan`	not-a-number (Float); nan never equals anything, itself included

Array utilities

Signature	Description
`unique(A)`	sorted distinct elements; vectors keep orientation, matrices flatten to a row
`sort(A)`	ascending sort: a vector as a whole, a matrix by column
`find(mask)`	1-based positions of nonzero/true elements (row-major)
`pick(mask, a, b)`	elementwise select: a where the mask is true, else b
`cumsum(A)`	cumulative sum along a vector, or down each column
`cumprod(A)`	cumulative product along a vector, or down each column
`diff(A)`	consecutive differences along a vector, or down each column
`flipud(A)`	reverse row order (flip up-down)
`fliplr(A)`	reverse column order (flip left-right)

Mathematical functions

Signature	Description
`abs(x)`	absolute value, or complex magnitude
`sqrt(x)`	square root (complex result for negative reals)
`cbrt(x)`	real cube root
`exp(x)`	e raised to the x (complex-aware)
`log(x)`	natural logarithm (complex for negatives)

Signature	Description
`ln(x)`	natural logarithm (alias for log)
`log10(x)`	base-10 logarithm (complex for negatives)
`log2(x)`	base-2 logarithm (complex for negatives)
`sign(x)`	-1 / 0 / +1 by sign; z/|z| for complex
`floor(x)`	round toward -infinity (componentwise on complex)
`ceil(x)`	round toward +infinity (componentwise on complex)
`round(x)`	round to nearest (componentwise on complex)
`trunc(x)`	round toward zero
`hypot(a, b)`	sqrt(a^2 + b^2) without overflow (elementwise)
`mod(a, b)`	modulo, result takes the sign of b (elementwise)
`rem(a, b)`	remainder, result takes the sign of a (elementwise)
`gamma(x)`	gamma function (real, elementwise)
`erf(x)`	error function (real, elementwise)
`erfc(x)`	complementary error function 1 - erf(x)
`beta(a, b)`	beta function (a, b > 0, elementwise)
`lbeta(a, b)`	log of the beta function
`gammainc(x, a)`	regularized lower incomplete gamma P(a, x) (the chi^2 CDF)
`betainc(x, a, b)`	regularized incomplete beta I_x(a, b) (Student-t / F CDFs)
`norminv(p)`	standard normal quantile (inverse CDF)
`digamma(x)`	digamma psi(x) = d/dx log gamma(x)
`besselj(n, x)`	Bessel function of the first kind, integer order n
`bessely(n, x)`	Bessel function of the second kind, integer order n (x > 0)
`lgamma(x)`	log of |gamma(x)| (real, elementwise)

Linear algebra

Signature	Description
`kron(A, B)`	Kronecker product: (m x n) kron (p x q) -> (mp x nq)
`dot(a, b)`	inner product of two vectors
`norm(x) | norm(x, p)`	vector p-norm (p = 1 or 2, default 2); matrix Frobenius norm
`trace(A)`	sum of the diagonal
`det(A)`	determinant via LU
`inv(A)`	matrix inverse (solves A I)
`lu(A)`	LU with partial pivoting -> {L, U, p}, so PA = LU
`qr(A)`	Householder QR -> {Q, R} (real or complex)
`chol(A)`	Cholesky factor L (lower), L*L' = A (SPD / Hermitian PD)
`eig(A)`	eigendecomposition -> {values, vectors}; Hermitian (ascending real) or general (complex)
`svd(A)`	thin SVD -> {U, S, V}, A = U*diag(S)*V' (S descending)

autodiff

Signature	Description
`dual(a, b)`	the dual number a + b*eps with eps^2 = 0 (elementwise; dual(x, seed) seeds a derivative direction)
`dualval(x)`	the value part of a dual; a plain number passes through (total, so constant branches differentiate)
`dualeps(x)`	the eps (derivative) part of a dual; 0 for a plain number
`hdual(x, s1, s2)`	the hyper-dual x + s1*eps1 + s2*eps2 (optional 4th arg seeds eps1*eps2); one pass carries an exact mixed second partial
`hdualval(x)`	the value part of a hyper-dual; plain numbers pass through
`hdual12(x)`	the eps1*eps2 (second-derivative) part; 0 for a plain number

Trigonometric & hyperbolic

Signature	Description
`sin(x)`	sine (complex-aware, elementwise)
`cos(x)`	cosine (complex-aware, elementwise)
`tan(x)`	tangent (complex-aware, elementwise)
`asin(x)`	arcsine (complex outside [-1, 1])
`acos(x)`	arccosine (complex outside [-1, 1])
`atan(x)`	arctangent (complex-aware)
`atan2(y, x)`	two-argument arctangent (elementwise)
`sinh(x)`	hyperbolic sine (complex-aware)
`cosh(x)`	hyperbolic cosine (complex-aware)
`tanh(x)`	hyperbolic tangent (complex-aware)
`asinh(x)`	inverse hyperbolic sine (complex-aware)
`acosh(x)`	inverse hyperbolic cosine (complex below 1)
`atanh(x)`	inverse hyperbolic tangent (complex outside (-1, 1))

Complex accessors

Signature	Description
`real(z)`	real part (elementwise)
`imag(z)`	imaginary part (elementwise)
`conj(z)`	complex conjugate (elementwise)
`angle(z)`	argument atan2(im, re) (elementwise)
`arg(z)`	argument atan2(im, re) (alias for angle)

Random numbers

Signature	Description
`rng(seed)`	reseed the generator (xoshiro256**); same seed, same stream
`rand() | rand(n) | rand(r, c)`	uniform draws on [0, 1)
`randn() | randn(n) | randn(r, c)`	standard-normal draws

Signature	Description
`randi(imax[, r, c]) | randi([lo, hi], ...)`	uniform random integers

Predicates

Signature	Description
`isnan(x)`	elementwise test for NaN -> logical
`isinf(x)`	elementwise test for +/-Inf -> logical
`isfinite(x)`	elementwise test for a finite value -> logical

Higher-order functions

Signature	Description
`map(f, A)`	apply f to each element of A, returning an array of results

optimization

Signature	Description
`nlmin(f, x0, opts?)`	professional minimization (NLopt; exact dual gradients); opts: {alg = "slsqp"|"lbfgs"|"bobyqa"|"cobyla", eq, ineq, lb, ub, xtol, maxeval}

statistics

Signature	Description
`ismissing(x)`	true where a value is missing: NaN in numerics, "" in strings; composes with mask indexing (x[!ismissing(x)])
`dropmissing(t)`	remove every row of a record-of-columns where any column is missing (NaN or "")

18. Grammar summary

Reserved words: `let in fn if then else end true false null for while do break continue return`.

```
program    := statement*
statement  := 'let' NAME '=' expr            # binding (global at top level)
            | NAME '=' expr                  # assignment (walks scopes)
            | lvalue '[' indices ']' '=' expr
            | 'for' NAME '=' expr 'do' statement* 'end'
            | 'while' expr 'do' statement* 'end'
            | 'break' | 'continue' | 'return' expr?
            | expr
expr       := 'let' NAME '=' expr 'in' expr
            | 'if' expr 'then' expr ('else' expr)? 'end'
```

```
            | 'fn' NAME* '->' expr
            | '(' statement (';' statement)* ')'     # block expression
            | expr binop expr | unop expr | expr postfix
            | NAME | literal | '[' rows ']' | '{' fields '}'
            | expr '|>' expr | expr '|>>' expr | expr '~>' expr
            | expr relop expr relop expr ...   (* one direction; middles bound once *)
            | expr 'where' name '=' expr (',' name '=' expr)*
            | expr '[' name '=' expr ']' expr   (* index-bound reduction *)
postfix    := "'" | ".'" | '(' args ')' | '[' indices ']' | '.' NAME
```

Statements separate by newline or `;` (a trailing `;` also suppresses the REPL echo). Comments run from `#` or `%` to end of line.

Every example in this manual was executed against the current interpreter; the builtin reference is generated from the source. If you find a discrepancy, that is a bug — please report it.